

MCA - Semester I
Python Programming & Relational Database

Assignment 1 - Mini Project

**CONSOLE RECORD-MANAGEMENT
APPLICATION**

Submitted by	Deeya Mathur
Enrollment / Roll No.	150097926004
Submitted to	Rohan Gupta
Date of submission	21/09/2026
GitHub repository	github.com/mathur458/console-management-system

1. Introduction

1.1 Overview

The Console Record-Management Application is a menu-driven program written in Python that manages a collection of records from the command line. The user can add new records, view all stored records, search for a record by name, update an existing record and delete a record. All records are saved in a CSV file, so the data is still available the next time the program is run.

Each record holds four fields - Name, Age, Course and Email. The project brings together the main topics of the first part of the course : data types and variables, conditional statements and loops, functions, exception handling and file input/output - in one working application.

1.2 Problem Statement

Small collections of records such as student lists or contact lists are often maintained by hand, which is slow and error-prone. A program that keeps the records in a file and offers a simple menu for the common operations removes this manual effort. The problem addressed by this project is to design and build such a console application, making sure that it copes with invalid user input and file-related errors without crashing.

1.3 Objectives

- Provide a menu-driven console interface that keeps running until the user chooses to exit.
- Support the five basic record operations: add, view, search, update and delete.
- Use suitable Python data types and data structures to hold and process records.
- Organise the program into separate functions, one for each operation.
- Validate user input and handle runtime and file errors using exception handling.
- Store records permanently in a file using File I/O so that data survives after the program is closed.

1.4 Scope

The application is a single-user program that runs in a terminal. Records are kept in a plain-text CSV file and each record contains Name, Age, Course and Email. The project does not include a graphical interface, a database server or multi-user access; possible extensions are discussed in Section 9.

2. Technologies and Environment

Item	Used	Purpose
Programming language	Python 3	Development of the whole application
Standard library module	csv	Reading and writing records in CSV format
Data storage	records.csv	Permanent storage of records
User interface	Command line / terminal	Menu display and user input
Version control	Git and GitHub	Source code management and submission

The application needs only a working installation of Python 3. No third-party packages have to be installed, because the only module imported is the built-in csv module.

3. System Design

3.1 Program Flow

When the program starts it prints a welcome banner and enters an infinite loop that shows the menu and reads the user's choice. The choice is converted to an integer; if this fails, an error message is printed and the menu is shown again. A valid choice from 1 to 5 calls the corresponding function, and after the function finishes, control returns to the menu. Choice 6 ends the loop and the program terminates. Figure 3.1 summarises this flow.

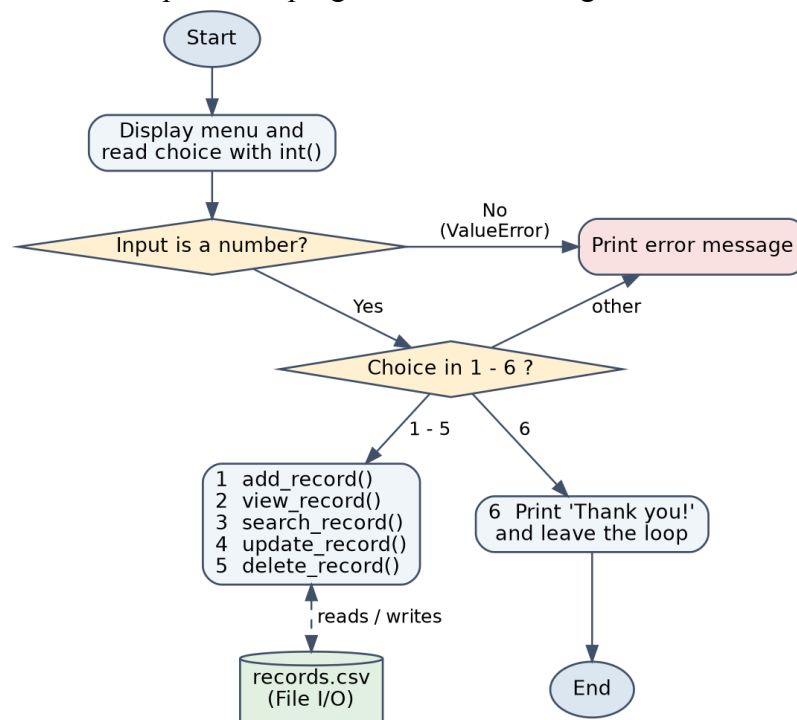


Figure 3.1: Overall program flow (after every operation or error message, control returns to the menu)

3.2 Data Design

Every record has four fields. In the file, a record is one line of comma-separated values. In memory, a record is a list of four items and a collection of records is a list of such lists.

Field	Type in program	Validation applied
Name	str	Must not be empty after removing leading/trailing spaces
Age	int (stored as text in CSV)	Must be a whole number greater than 0
Course	str	Must not be empty after removing leading/trailing spaces
Email	str	Must contain both "@" and "."

Example of a stored record (one line of records.csv):

Deeya Mathur,21,MCA,deeya@gmail.com

3.3 Function Design

Instead of writing all logic in one block, each operation is placed in its own function. This keeps every function short and focused and makes the program easier to read, test and modify.

Function	Responsibility
add_record()	Reads and validates Name, Age, Course and Email, then appends the record to the CSV file.
view_record()	Reads the CSV file and prints every stored record; reports when there are no records.
search_record()	Asks for a name and prints all records whose name matches, ignoring upper/lower case.
update_record()	Finds a record by name, reads new validated details from the user, and rewrites the file with the changed record.
delete_record()	Finds a record by name, removes it from the list of records, and rewrites the file.
Main menu loop	Displays the menu, validates the choice and calls the matching function until Exit is chosen.

3.4 Design Decisions

- Validate before saving. Input is checked before anything is written to the file, so an invalid entry never reaches the data file.
- Use the csv module. It handles separators and quoting correctly, so values such as names containing commas do not break the file format.

- Read - modify - rewrite for update and delete. A CSV file cannot be edited in the middle of a line, so all records are loaded into a list, the list is changed, and the file is written again.
- Skip malformed rows. Rows that do not have exactly four fields are ignored while reading, which stops one damaged line from crashing the whole program.

4. Implementation

The complete program is contained in one file, `record_management.py` (about 300 lines).

4.1 Import and Constant

The `csv` module is imported and the name of the data file is stored in the constant `FILE_NAME`, so the file name is defined in one place only.

```
import csv
FILE_NAME = 'records.csv'
```

4.2 Adding a Record - `add_record()`

The function reads the name, age, course and email. Age is converted with `int()` inside a try block; a non-numeric value raises `ValueError`, which is caught and reported, and a value of zero or below is rejected by an if statement. If any validation fails the function returns without touching the file.

```
try:
    age = int(input("Enter Age: "))

    if age <= 0:
        print("Age must be greater than 0.")
        return

except ValueError:
    print("Please enter a valid age.")
    return
```

After all fields pass validation, the record is appended to the file using mode "a". Because the file is opened with a with statement, it is closed automatically. Any operating-system error while saving is caught by except `OSError`.

```
try:
    with open(FILE_NAME, "a", newline="") as file:
        writer = csv.writer(file)
        writer.writerow([name, age, course, email])

    print("Record added successfully!")

except OSError:
    print("Error: Unable to save the record.")
```

4.3 Viewing Records - view_record()

The file is opened in read mode and each row is processed in a for loop. Rows that do not contain exactly four fields are skipped with continue. The Boolean variable found records whether at least one record was printed, so that "No records found." can be shown when the file is empty. If the file does not exist yet, FileNotFoundError is caught and the same friendly message is shown.

```
found = False

try:
    with open(FILE_NAME, "r", newline="") as file:
        reader = csv.reader(file)

        for row in reader:

            if len(row) != 4:
                continue

            print("-----")
            print("Name  :", row[0])
            print("Age   :", row[1])
            print("Course :", row[2])
            print("Email  :", row[3])

            found = True

        if not found:
            print("No records found.")

except FileNotFoundError:
    print("No records found.")

except OSError:
    print("Error: Unable to read the records file.")
```

4.4 Searching - search_record()

The name typed by the user is compared with the name in each row after both are converted to lower case, so the search is not case-sensitive (for example, "deeya mathur" finds "Deeya Mathur").

```
for row in reader:

    if len(row) != 4:
        continue

    if row[0].strip().lower() == search_name.lower():

        print("\nRecord Found!")
        print("Name  :", row[0])
        print("Age   :", row[1])
```

```
print("Course :", row[2])
print("Email :", row[3])

found = True
```

4.5 Updating - update_record()

The function first loads all valid rows into the list records.

```
records = []

try:
    with open(FILE_NAME, "r", newline="") as file:
        reader = csv.reader(file)

        for row in reader:

            if len(row) == 4:
                records.append(row)

except FileNotFoundError:
    print("No records found.")
    return

except OSError:
    print("Error: Unable to read the records file.")
```

It then searches the list for the requested name, asks for the new details (validated in the same way as when adding), and replaces the values in that row. Note that age is converted back to a string because the list holds text read from the file.

```
row[0] = new_name
row[1] = str(new_age)
row[2] = new_course
row[3] = new_email

found = True
break
```

Finally the whole list is written back using mode "w", which replaces the old file content with the updated data.

```
try:
    with open(FILE_NAME, "w", newline="") as file:
        writer = csv.writer(file)
        writer.writerows(records)

    print("Record updated successfully!")

except OSError:
    print("Error: Unable to update the record.")
```

4.6 Deleting - delete_record()

Deletion follows the same read - modify - rewrite pattern as updating. The matching record is removed from the list with remove() and the remaining records are written back to the file.

for row in records:

```
    if row[0].strip().lower() == search_name.lower():
        records.remove(row)
        found = True
        break
```

4.7 Main Menu Loop

The menu is displayed inside a while True loop. The choice is converted with int() inside try/except so that non-numeric input does not stop the program. An if / elif / else chain selects the function to call, and choice 6 uses break to leave the loop.

Main Menu

while True:

```
    print("""
What would you like to do?
```

```
1. Add Record
2. View Record
3. Search Record
4. Update Record
5. Delete Record
6. Exit
""")
```

```
    try:
        n = int(input("Enter Number: "))
```

```
    except ValueError:
        print("Please enter a number from 1 to 6.")
        continue
```

```
    if n == 1:
        add_record()
```

```
    elif n == 2:
        view_record()
```

```
    elif n == 3:
        search_record()
```

```
    elif n == 4:
        update_record()
```

```

elif n == 5:
    delete_record()

elif n == 6:
    print("Thank you!")
    break

else:
    print("Invalid choice. Please enter a number from 1 to 6.")

```

5. Python Concepts Used

The table below maps each concept required by the assignment to its use in the application.

Concept	How it is used in the project
Data types and variables	str for name, course and email; int for age and the menu choice; bool (found) to track whether a record was located; list for a single record and list of lists (records) for the collection; constant FILE_NAME for the file name.
Conditional statements	if / elif / else chain in the main menu; if checks for empty input, age $\leq$ 0, invalid e-mail and for matching names during search, update and delete.
Loops	while True keeps the menu running until Exit; for loops read rows from the CSV file and iterate over the records list.
Functions	Separate functions add_record(), view_record(), search_record(), update_record() and delete_record(), each performing one task.
Exception handling	try / except ValueError for non-numeric input; except FileNotFoundError for a missing data file; except OSError for other file problems.
File I/O	Files opened with open() inside with blocks in modes "a", "r" and "w"; records read and written using csv.reader and csv.writer.
Menu-driven design	A six-option console menu that is redisplayed after every operation.

6. Exception Handling

The application never assumes that the user will type correct data or that the file system will always work. Two kinds of protection are used: try / except blocks for run-time errors, and if-based checks for values that are the right type but not acceptable.

Situation	Handled by	Message shown to the user
Menu choice is not a number	except ValueError	Please enter a number from 1 to 6.
Menu choice is a number outside 1-6	else branch	Invalid choice. Please enter a number from 1 to 6.
Age is not a number	except ValueError	Please enter a valid age.
Age is zero or negative	if age <= 0	Age must be greater than 0.
Name is empty	if name == ""	Name cannot be empty.
Course is empty	if course == ""	Course cannot be empty.
E-mail has no "@" or "."	if check	Please enter a valid email.
records.csv does not exist yet	except FileNotFoundError	No records found.
File cannot be read	except OSError	Error: Unable to read the records file.
File cannot be written	except OSError	Error: Unable to save / update / delete the record.

FileNotFoundError is a subclass of OSError, so in the functions that use both, the except FileNotFoundError clause is written first. If the order were reversed, the more general OSError would catch the missing-file case as well and the friendly "No records found." message would never appear.

7. File Handling and Data Persistence

Records are stored in records.csv in the same folder as the program. The file is created automatically the first time a record is added, so no manual setup is needed. Three file modes are used:

Mode	Used in	Purpose
"a"	add_record()	Append a new line at the end without disturbing existing records
"r"	view, search, update, delete	Read existing records from the file
"w"	update_record(), delete_record()	Rewrite the whole file after the list of records has been changed

Every file is opened with a with statement, which closes the file automatically even if an error occurs. The files are opened with newline="" as recommended in the Python documentation for the csv module, so line endings are handled correctly on all operating systems.

Because the data lives in a file rather than in memory, it is not lost when the program exits. This was verified in testing (test case TC18): a record added in one run of the program was still listed when the program was started again.

8. Testing and Results

8.1 Test Approach

The application was run on Windows using Python 3.13 from the terminal; the screenshots in Section 8.3 were taken from these runs. The test cases in Section 8.2 cover normal operation of every menu option, invalid input, missing records and file-related errors. Each case was executed by running `python record_management.py` and supplying the listed inputs at the prompts, and the test cases were also repeated with scripted input under Python 3.12. For TC19 and TC20 a file error was simulated by making `records.csv` unusable (a folder with that name was created), which forces the `open()` call to fail.

8.2 Test Cases and Results

ID	Scenario	Input	Observed output	Result
TC01	View when data file does not exist	2	No records found.	Pass
TC02	Add a valid record	1 → Asha Verma, 24, MCA, asha@example.com	Record added successfully!	Pass
TC03	Add a second valid record	1 → Rohan Iyer, 22, BCA, rohan@example.com	Record added successfully!	Pass
TC04	View all records	2	Both records printed with Name, Age, Course and Email	Pass
TC05	Search, different letter case	3 → asha verma	Record Found! Details of Asha Verma displayed	Pass
TC06	Search for a name that does not exist	3 → Nobody	Record not found.	Pass
TC07	Update an existing record	4 → Rohan Iyer; new: Rohan Iyer, 23, MCA, rohan.iyer@example.com	Record updated successfully!; View shows the new values	Pass
TC08	Delete an existing record	5 → Asha Verma	Record deleted successfully!; View shows only Rohan Iyer	Pass
TC09	Update a record that does not exist	4 → Ghost	Record not found.	Pass
TC10	Delete a record that does not exist	5 → Ghost	Record not found.	Pass
TC11	Age is not a number	Age: abc	Please enter a valid age.	Pass
TC12	Age is negative	Age: -5	Age must be greater than 0.	Pass
TC13	Empty name	Name: (press Enter)	Name cannot be empty.	Pass

ID	Scenario	Input	Observed output	Result
TC14	Empty course	Course: (press Enter)	Course cannot be empty.	Pass
TC15	Invalid e-mail	Email: deeya	Please enter a valid email.	Pass
TC16	Menu choice is text	Enter Number: abc	Please enter a number from 1 to 6.	Pass
TC17	Menu choice out of range	Enter Number: 8	Invalid choice. Please enter a number from 1 to 6.	Pass
TC18	Data persists after restart	Add record → Exit → run again → View	Record is still listed	Pass
TC19	File error while saving (records.csv made unusable)	1 → valid details	Error: Unable to save the record.	Pass
TC20	File error while reading (records.csv made unusable)	2	Error: Unable to read the records file.	Pass
TC21	Exit the program	6	Thank you! Program ends normally	Pass

All 21 test cases produced the expected output, and the program did not terminate unexpectedly in any of them.

8.3 Screenshots

The following screenshots show the application running.

```

===== Welcome to Console Record Management System by Deeya Mathur =====

What would you like to do?

1. Add Record
2. View Record
3. Search Record
4. Update Record
5. Delete Record
6. Exit

Enter Number: █

```

Figure 8.1: Main menu

```
Enter Number: 1

----- Add Record -----
Enter Name: Deeya Mathur
Enter Age: 21
Enter Course: MCA
Enter Email: deeyamathur@gmail.com
Record added successfully!
```

Figure 8.2: Adding a record

```
Enter Number: 2

----- View Records -----
-----
Name    : Deeya Mathur
Age     : 21
Course  : MCA
Email   : deeyamathur@gmail.com
```

Figure 8.3: Viewing records

```
Enter Number: 3

----- Search Record -----
Enter Name to search: Deeya Mathur

Record Found!
Name    : Deeya Mathur
Age     : 21
Course  : MCA
Email   : deeyamathur@gmail.com
```

Figure 8.4: Searching for a record

```
Enter Number: 4

----- Update Record -----
Enter Name to update: Deeya Mathur

Enter new details:
Enter Name: Deeya Mathur
Enter Age: 23
Enter Course: MSc
Enter Email: deeyamathur@gmail.com
Record updated successfully!
```

Figure 8.5: Updating a record

```
Enter Number: 5

----- Delete Record -----
Enter Name to delete: Deeya Mathur
Record deleted successfully!
```

Figure 8.6: Deleting a record

```
Enter Number: 1

----- Add Record -----
Enter Name: Deeya Mathur
Enter Age: 21
Enter Course: MCA
Enter Email: mathur123
Please enter a valid email.
```

Figure 8.7: Invalid e-mail is rejected

```
Enter Number: 4

----- Update Record -----
Enter Name to update: Deeya Mathur
Record not found.
```

Figure 8.8: Searching for a record that does not exist

```
Enter Number: 6
Thank you!
PS C:\Users\stu66\Downloads\console-management-system>
```

Figure 8.9: Exiting the application

9. Limitations and Future Enhancements

9.1 Current Limitations

- Records are identified only by name. If two records have the same name, update and delete act on the first match only.
- Name matching is exact (apart from letter case); partial names such as "Asha" will not find "Asha Verma".
- E-mail validation is basic: it only checks for the presence of "@" and ".".
- Rows that do not contain exactly four fields are ignored when reading and are therefore not written back when the file is rewritten by update or delete.
- Pressing Ctrl+C or Ctrl+D at a prompt ends the program with a Python error message, because these interruptions are not handled.
- The data is stored as plain text without any protection or indexing, which is suitable only for small amounts of data.

9.2 Possible Enhancements

- Give every record a unique ID so that records with the same name can be told apart.
- Add partial-match search and search by other fields such as course.
- Handle KeyboardInterrupt and EOFError so the program exits cleanly, and place the menu loop under an `if __name__ == "__main__":` guard with a `main()` function.
- Improve validation using regular expressions for e-mail addresses.
- Store the data in a relational database such as SQLite or MySQL and use SQL queries for the operations. This would connect the project to the Relational Database part of the course.

10. Conclusion

The Console Record-Management Application meets the requirements of the assignment. It offers a menu-driven interface, supports adding, viewing, searching, updating and deleting records, uses functions to keep the code organised, validates input with exception handling, and stores data in a CSV file so that it persists between runs.

Building the project showed how the individual topics of the course - variables and data types, conditions, loops, functions, exceptions and file handling - work together in a complete program. It also showed the value of validating input early and of separating a program into small functions. The limitations listed in Section 9 provide clear directions for extending the project in later work.

References

- [1] Python Software Foundation. csv - CSV File Reading and Writing. Python 3 documentation. <https://docs.python.org/3/library/csv.html>
- [2] Python Software Foundation. Reading and Writing Files. The Python Tutorial. <https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files>
- [3] Python Software Foundation. Errors and Exceptions. The Python Tutorial. <https://docs.python.org/3/tutorial/errors.html>
- [4] Project source code and documentation, GitHub repository. <https://github.com/mathur458/console-management-system>

Appendix A - Complete Source Code

File: record_management.py

```
1 import csv
2
3 FILE_NAME = "records.csv"
4
5 print("===== Welcome to Console Record Management System by Deeya Mathur
=====")
6
7
8 # Add Record
9 def add_record():
10     print("\n----- Add Record -----")
11
12     name = input("Enter Name: ").strip()
13
14     if name == "":
15         print("Name cannot be empty.")
16         return
17
18     try:
19         age = int(input("Enter Age: "))
20
21         if age <= 0:
22             print("Age must be greater than 0.")
23             return
24
25     except ValueError:
26         print("Please enter a valid age.")
27         return
28
29     course = input("Enter Course: ").strip()
30
31     if course == "":
32         print("Course cannot be empty.")
33         return
34
35     email = input("Enter Email: ").strip()
36
37     if "@" not in email or "." not in email:
38         print("Please enter a valid email.")
39         return
40
41     try:
42         with open(FILE_NAME, "a", newline="") as file:
43             writer = csv.writer(file)
44             writer.writerow([name, age, course, email])
45
46     print("Record added successfully!")
47
```

```
48     except OSError:
49         print("Error: Unable to save the record.")
50
51
52 # View Records
53 def view_record():
54     print("\n----- View Records -----")
55
56     found = False
57
58     try:
59         with open(FILE_NAME, "r", newline="") as file:
60             reader = csv.reader(file)
61
62             for row in reader:
63
64                 if len(row) != 4:
65                     continue
66
67                 print("-----")
68                 print("Name   :", row[0])
69                 print("Age    :", row[1])
70                 print("Course :", row[2])
71                 print("Email  :", row[3])
72
73                 found = True
74
75     if not found:
76         print("No records found.")
77
78     except FileNotFoundError:
79         print("No records found.")
80
81     except OSError:
82         print("Error: Unable to read the records file.")
83
84
85 # Search Record
86 def search_record():
87     print("\n----- Search Record -----")
88
89     search_name = input("Enter Name to search: ").strip()
90
91     if search_name == "":
92         print("Please enter a name.")
93         return
94
95     found = False
96
97     try:
98         with open(FILE_NAME, "r", newline="") as file:
```

```
99     reader = csv.reader(file)
100
101     for row in reader:
102
103         if len(row) != 4:
104             continue
105
106         if row[0].strip().lower() == search_name.lower():
107
108             print("\nRecord Found!")
109             print("Name  :", row[0])
110             print("Age   :", row[1])
111             print("Course :", row[2])
112             print("Email :", row[3])
113
114             found = True
115
116         if not found:
117             print("Record not found.")
118
119     except FileNotFoundError:
120         print("No records found.")
121
122     except OSError:
123         print("Error: Unable to read the records file.")
124
125
126 # Update Record
127 def update_record():
128     print("\n----- Update Record -----")
129
130     search_name = input("Enter Name to update: ").strip()
131
132     if search_name == "":
133         print("Please enter a name.")
134         return
135
136     records = []
137
138     try:
139         with open(FILE_NAME, "r", newline="") as file:
140             reader = csv.reader(file)
141
142             for row in reader:
143
144                 if len(row) == 4:
145                     records.append(row)
146
147     except FileNotFoundError:
148         print("No records found.")
149     return
```

```
150
151 except OSError:
152     print("Error: Unable to read the records file.")
153     return
154
155 found = False
156
157 for row in records:
158
159     if row[0].strip().lower() == search_name.lower():
160
161         print("\nEnter new details:")
162
163         new_name = input("Enter Name: ").strip()
164
165         if new_name == "":
166             print("Name cannot be empty.")
167             return
168
169         try:
170             new_age = int(input("Enter Age: "))
171
172             if new_age <= 0:
173                 print("Age must be greater than 0.")
174                 return
175
176         except ValueError:
177             print("Please enter a valid age.")
178             return
179
180         new_course = input("Enter Course: ").strip()
181
182         if new_course == "":
183             print("Course cannot be empty.")
184             return
185
186         new_email = input("Enter Email: ").strip()
187
188         if "@" not in new_email or "." not in new_email:
189             print("Please enter a valid email.")
190             return
191
192         row[0] = new_name
193         row[1] = str(new_age)
194         row[2] = new_course
195         row[3] = new_email
196
197         found = True
198         break
199
200 if not found:
```

```
201     print("Record not found.")
202     return
203
204     try:
205         with open(FILE_NAME, "w", newline="") as file:
206             writer = csv.writer(file)
207             writer.writerows(records)
208
209         print("Record updated successfully!")
210
211     except OSError:
212         print("Error: Unable to update the record.")
213
214
215 # Delete Record
216 def delete_record():
217     print("\n----- Delete Record -----")
218
219     search_name = input("Enter Name to delete: ").strip()
220
221     if search_name == "":
222         print("Please enter a name.")
223         return
224
225     records = []
226
227     try:
228         with open(FILE_NAME, "r", newline="") as file:
229             reader = csv.reader(file)
230
231             for row in reader:
232
233                 if len(row) == 4:
234                     records.append(row)
235
236     except FileNotFoundError:
237         print("No records found.")
238         return
239
240     except OSError:
241         print("Error: Unable to read the records file.")
242         return
243
244     found = False
245
246     for row in records:
247
248         if row[0].strip().lower() == search_name.lower():
249             records.remove(row)
250             found = True
251             break
```

```
252
253     if not found:
254         print("Record not found.")
255         return
256
257     try:
258         with open(FILE_NAME, "w", newline="") as file:
259             writer = csv.writer(file)
260             writer.writerows(records)
261
262         print("Record deleted successfully!")
263
264     except OSError:
265         print("Error: Unable to delete the record.")
266
267
268 # Main Menu
269 while True:
270
271     print("""
272 What would you like to do?
273
274 1. Add Record
275 2. View Record
276 3. Search Record
277 4. Update Record
278 5. Delete Record
279 6. Exit
280 """)
281
282     try:
283         n = int(input("Enter Number: "))
284
285     except ValueError:
286         print("Please enter a number from 1 to 6.")
287         continue
288
289     if n == 1:
290         add_record()
291
292     elif n == 2:
293         view_record()
294
295     elif n == 3:
296         search_record()
297
298     elif n == 4:
299         update_record()
300
301     elif n == 5:
302         delete_record()
```

```
303
304     elif n == 6:
305         print("Thank you!")
306         break
307
308     else:
309         print("Invalid choice. Please enter a number from 1 to 6.")
```

Appendix B - Repository Details and How to Run

B.1 Repository Contents

```
console-management-system/
|-- record_management.py  (application source code)
|-- records.csv          (data file, created on first add)
|-- README.md            (project overview and usage)
|-- Assignment_Report.docx (this report)
`-- screenshots/         (screenshots of the running program)
```

B.2 How to Run

- Install Python 3 (no additional packages are required).
- Clone the repository: `git clone https://github.com/mathur458/console-management-system.git`
- Open the project folder: `cd console-management-system`
- Run the program: `python record_management.py` (or `python3 record_management.py` on some systems)
- Choose an option from the menu by typing its number. Choose 6 to exit.